The Loop and the Directive

5 August 2026

A founder directive arrived at 17:46 as a Word document. Forty-nine minutes later it was bound to the autonomous loop by content hash. Two hours after that it had refuted two modules that had never been attacked.

This document is the operational record: what the directive changed, what worked, and — in the same detail — what did not.

THE DAY IN NUMBERS. 21 fires · 11 refutations · 10 survivals · 3 directive-bound · 1 hour lost to a defect I introduced · 46 obligations now tracked with an owner

THE UNCOMFORTABLE PART. An adversarial review of this same day's engineering found twelve defects in it, including a commit message asserting a guard that was never written. Those are reported here at the same resolution as the successes, because a record that only contains successes cannot be checked.

EXECUTIVE SUMMARY

What the directive changed

In one sentence. The loop went from doing good work that nobody could trace to a stated obligation, to doing work that names the obligation it serves, cites the document by SHA-256, and fails closed if it cannot.

BEFORE

Campaign items carried an objective and a done criterion. Nothing linked them to anything the founder had asked for. No receipt cited a directive. No report could answer "which of the founder's asks are closed?" — and Sections B and C of the directive, which gate most of the portfolio's remaining upside, were invisible to the loop's accounting entirely.

AFTER

- 46 obligations indexed, each with an owner class (25 loop, 10 human, 11 lab) and a machine-checkable mapping to campaign items.
- 48 items carry a `directive_ref`; every fire echoes it; every receipt must cite the document by path and hash.
- Six gates now enforce in code what used to be enforced by prose.
- A coverage report exits non-zero if any obligation loses its owner or any item cites a directive that does not resolve.

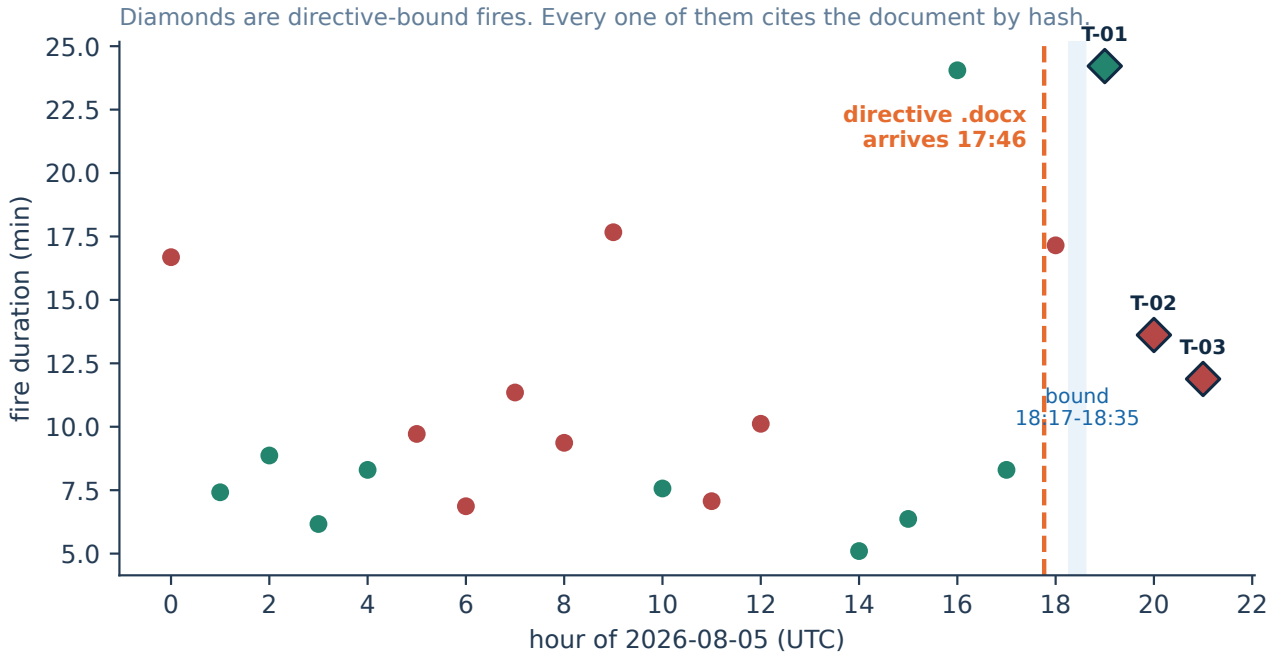
AND IT IMMEDIATELY EARNED ITS KEEP

Three directive probes have run. Two refuted their targets — each against a criterion written by the founder before the attack existed. The modules involved (`cro-protocol-packager`, `envelope-gates`) had carried composite scores for weeks without ever meeting an adversary.

Red team. One caveat on every number above: 6 of 46 obligations are marked closed, and "closed" means a signed receipt matched a glob. It does not mean a human agreed the founder's intent was served. The audit that checks exactly that is queued and has not run.

TIMELINE

A Word document to a refuted module in 3h



17:46 The directive arrives

43,714 bytes, 519 lines, 40 obligations across three owner classes. It lands as a .docx in a home directory — referenced by nothing, readable by no tool in the loop.

18:09 Rendered, indexed, placed

Converted to Markdown deterministically (zipfile + regex, no new dependency, byte-identical across replays, so the hash is stable and citable). Indexed into 46 machine-readable obligations, with the twelve Section A.1 probes PARSED out of the rendering rather than transcribed, so their criteria cannot drift. Canonical copy placed on xbabe2 in the evidence tree; hashes verified identical on both hosts.

18:17 Bound to the loop

directive_ref wired at all three points it must exist to survive: the prompt schema, the launcher's parser, and the summary writer's whitelist. The authoritative value is read from the campaign item, never from the agent's echo — the echo is recorded beside it so a fire reporting the wrong obligation stays visible.

18:20 Priority rewritten

The directive's twelve first-attack probes merged INTO the existing items rather than filed as duplicates, and promoted to the head of the queue. This deliberately paused the conformal-repair work; the founder's stated order won.

18:22 Human and lab work made visible

21 obligations the loop is forbidden from executing filed as blocked items with owner classes, so they appear in the same accounting instead of living outside it.

18:30 Version controlled

The evidence tree's hourly autocommit picked the directive up as commit 80bb28e. Durable with no manual git step.

19:00 T-01 — first directive-bound fire

candidate-contract, which had never been attacked. Eight injection encodings, 0 reached a typed field, byte-identical to the clean control across all of them. SURVIVED — with a positive control proving the detector fires when the defect is present, a verbatim restatement of the founder's criterion, and an independent re-hash of the directive that nobody asked it to do.

20:00 T-02 — the directive pays for itself

cro-protocol-packager REFUTED. A packet-authored decision threshold with a format-valid but content-underived lock hash passes validate, sign and verify end-to-end. The lock is checked for shape, never recomputed from what it locks.

20:23–20:35 Red-team gap closure

An adversarial review of the day's own engineering returned twelve defects. Five phases of fixes deployed in twelve minutes, each runtime-tested before deploy.

21:00 T-03 — and again

envelope-gates REFUTED. An 18-cell metamorphic boundary grid found five pass-on-invalid-input slips: a value-free $\pm\infty$ bypass on a real registered policy, NaN-feature masking through Python's non-total-order min(), and no policy-provenance checking at all. All six new gates reported healthy on the same fire.

The hour that is missing. There is no fire at 13:00. That is not an idle hour. A fire ran, did its work correctly, and signed its receipt — and then the launcher aborted before recording anything, because I had referenced a shell variable nineteen lines before it was assigned. Under `set -u` that is fatal. The work exists on xbabe2; the record of it does not. `bash -n` validates syntax and cannot see execution order, which is why a dedicated check now exists.

WHAT WORKED

Verified, not asserted

THE BINDING HOLDS END TO END

- Every directive-bound fire carried `directive_ref` into its summary record, with the authoritative value and the agent's echo matching exactly.
- T-01's receipt cites `directive_path`, `directive_sha256` and `directive_obligation`, and restates the founder's pre-committed refutation criterion verbatim across three fields — then adds `directive_sha256_verified_now`, an independent re-hash of the document, which no instruction requested.
- The drift check reported `directive: OK` on every fire after deployment.
- The backlog alarm fired correctly, naming eleven modules one fire from empty.

THE DIRECTIVE FOUND REAL DEFECTS, FAST

Two of three. Three probes have run. Two refuted their targets against criteria the founder pre-committed. Both modules had zero prior verdicts and non-trivial composite scores — which is precisely the liability the directive identified: an unattacked score measures evidence density, not correctness.

THE POSITIVE-CONTROL GATE WORKS

Deployed at 12:30 after the loop attacked its own quality bar and won — a fire built an attack whose refutation criterion was algebraically unreachable and it passed every check. Since then a survival is credited only if the fire planted a defect of the class its attack exists to catch and showed the attack fired on it. The 14:00 fire planted two (a front-matter tamper and a ceiling tamper) and detected both.

WHAT DID NOT WORK

Reported at the same resolution

A false claim in the permanent record. A commit message stated that "a validator now asserts that no HUMAN- or LAB-owned item is ever pending". **No such validator was committed.** The assertion lived in a single-use generator script that ran once from a scratch directory and never entered the repository. The only real protection was that all 21 such items happened to be blocked — a data state, not an invariant. It is recorded in `directives/CORRECTIONS.md`, and both the validator and a runtime guard now exist so the claim is true in both senses it implied.

THE LESSON THAT WAS LEARNED AND THEN NOT APPLIED

At 12:30 the survival bar was moved from prose into code, because prose enforcement had just been proven gameable. At 18:17 a new mechanism was shipped — the directive citation — enforced only by prose. A fire could have omitted every directive field and still closed its item. It was found by adversarial review at 20:23 and gated by 20:27. The gap was under two hours, but the point stands: the lesson was available and was not transferred.

DEFECTS FOUND IN THE DAY'S OWN ENGINEERING

- The index misreported 17 of 45 records as unfiled for obligations whose items already existed. It is now DERIVED from the campaign rather than hand-asserted, so the failure mode is removed rather than corrected once.
- Section C.0's promotion gate and kill/hold gate — eight criteria governing every lab experiment, including "on failure no threshold is weakened and no fixture is substituted" — were dropped from the index entirely.
- Newly filed lab items `C-1..C-10` collided with pre-existing `C1..C10`. Worse, two were semantic duplicates: the legacy items were the same rights asks as `R-D01` and `R-D03`. Renamed and cross-linked.
- The document hash was duplicated across the prompt, the index and 48 items with nothing comparing them. The prompt now takes it from the index by placeholder.

AND ONE CAUGHT BY THE TOOLING BUILT THAT MORNING

While writing the drift check — whose entire purpose is preventing one kind of inconsistency — I reintroduced the use-before-assignment bug that had broken the 13:00 fire. The check written in response to that bug caught it before deploy.

THE SIX GATES

Enforced by code, not instruction

gate	what it makes impossible
`OWNER_GUARD_SKIP`	The loop dispatching a fire against an obligation the founder or a lab must own — whatever that item's status says.
`directive_citation`	A verdict closing an item without proving which founder obligation it answers. Matches on the document hash, the obligation id and the founder's own sentence — **not on field names**, because a hand review of T-01 wrongly reported its criterion absent by searching one literal key.
`positive_control`	A survival whose attack could never have failed.
`DIRECTIVE_DRIFT`	Banking a verdict against a document the loop cannot prove it is reading. Re-hashes the live file every fire; a mismatch blocks acceptance.
`DIRECTIVE_REF_MISMATCH`	A fire silently working on a different obligation than the one it was handed.
`MODULE_EMPTY` / `BACKLOG_LOW`	A module going idle without anyone noticing.

TWO DESIGN CHOICES WORTH STATING

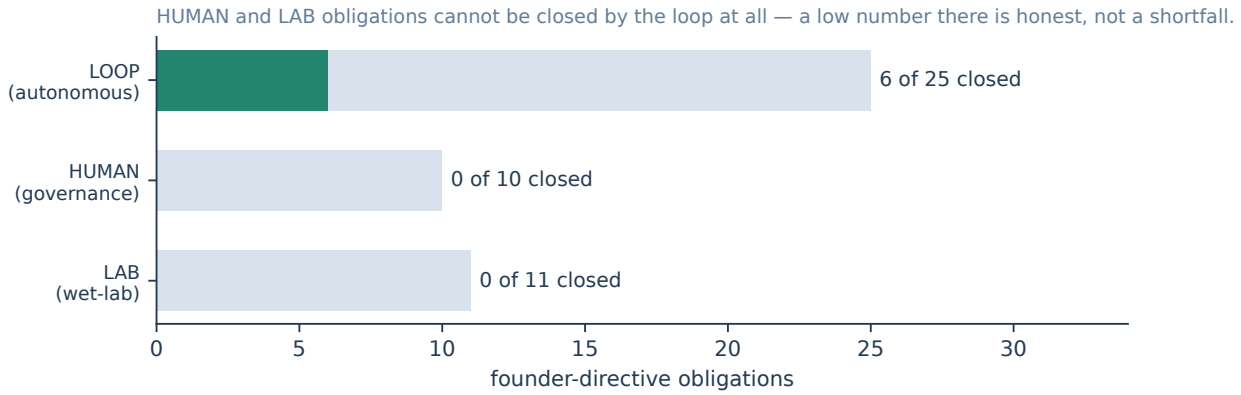
- **Both content gates are non-saturating.** A downgrade does not count toward the no-progress counter, because such a fire still banked a verified artifact — it simply has not earned the word it claimed. Without that, three honest survivals in a row would stop the loop.
- **Neither gate re-executes agent code.** They validate what a receipt records. Re-execution would turn the verifier into a remote-execution surface, which is a worse problem than the one it solves.

BOTH CHECKERS HAVE POSITIVE CONTROLS

A checker that has only ever passed is indistinguishable from no checker — the criticism the loop levels at a schema guard that has never fired. So the invariants validator was run against a copy with six planted defects (a human item and a lab item flipped to pending, a glob collision, a wrong document hash, a ghost obligation, an over-budget item) and caught all six. The coverage tool was run against three planted defects and caught all three — and then immediately caught a real one in its author's own work.

ACCOUNTABILITY

Index, reference, report



THREE ARTIFACTS, ONE CHAIN

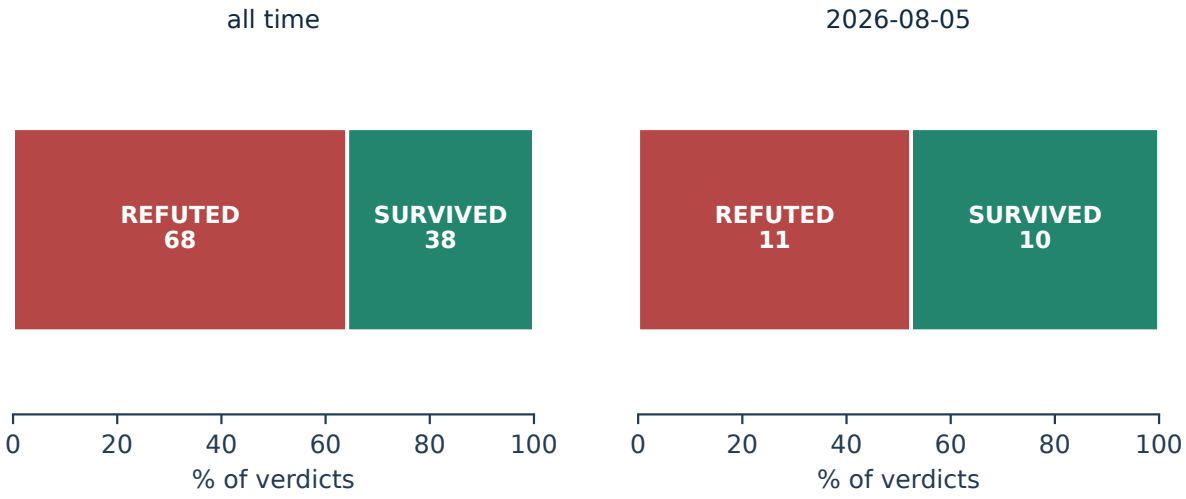
1. **The index** — 46 obligations, each with an owner, a section, and where the founder wrote one, a verbatim done criterion and refutation criterion. The Section A probes are PARSED from the rendered document, not transcribed, so they cannot drift from the source. Every record carries an `origin` field distinguishing founder text from the 2 obligations we added around it, so our additions can never be read back as the founder's asks.
2. **The reference** — 48 campaign items carry a `directive_ref`. The launcher takes the authoritative value from the item, not from the agent, and records the agent's echo beside it.
3. **The report** — joins the two against the fire log and fails on either of the two conditions that matter: an obligation with no campaign item behind it, or an item citing a directive that does not resolve.

WHERE WE KNOWINGLY DIVERGED FROM THE DIRECTIVE

Recorded in the index itself, not in a commit message, because a directive we quietly diverge from is worse than no directive:

- The heartbeat obligation was already delivered that morning; not re-filed.
- Two of the three priority verification items were already satisfied.
- The directive states that 15 of 26 repairs carry no independent verify. Re-derived from the live file the figure is about 8 — the count came from a name-prefix scan that misses five verify items using a different naming convention. Filing 15 new items would have duplicated existing work.

VERDICT MIX



Today's mix is more refutation-heavy than the all-time mix. That is what directing fires at never-attacked modules does, and it is the intended effect: the cheapest defects to find are in the code nobody has attacked yet.

INTERNALS

Two hosts, one fire an hour

WHERE THINGS RUN

Split by design. **xbabe1** runs the cron and holds the campaign, the prompt and the launcher. **xbabe2** holds all code, all data and all evidence. The agent runs on xbabe1 and performs every real action over `ssh xbabe2`. xbabe1 never mirrors the evidence tree, so there is exactly one place a receipt can live.

ONE FIRE, END TO END

1. **Lock.** `flock` on a fixed path. One fire at a time, ever.
2. **Heartbeat.** A line is written before anything can fail, so an hour where the agent never starts leaves a trace rather than a silence.
3. **Hard-stop check.** If a previous fire tripped a protected-file guard, stop.
4. **Pick.** First pending item wins — file order is priority. Human- and lab-owned items are skipped whatever their status.
5. **Drift check.** Re-hash the founder directive on xbabe2 and compare to the index. A mismatch blocks verdict acceptance.
6. **Pre-fire snapshot.** Content hashes of the target module, plus a hash of the ledger prefix, so attribution is by content and history cannot be rewritten.
7. **Fire.** The agent runs with a pinned model against a rendered prompt.
8. **Verify.** Set-diff the snapshot, intersect with the artifacts the agent DECLARED, verify each signature for real, confirm the ledger line carries this fire's id, check the positive control and the directive citation.
9. **Accept or downgrade.** A verdict unsupported by verified evidence becomes `UNKNOWN` with a reason the next fire on that item will read.
10. **Update, mirror, prune.** Campaign written atomically; log pushed to xbabe2.

THE SCRIPTS

file	lines	what it does
`claude-hourly-loop.sh`	585	The launcher. Picks the item, snapshots the target, fires the agent, verifies the result, updates the campaign, mirrors the log.
`lib/remote_verify.py`	387	Runs ON xbabe2. Content-hash snapshot, real Ed25519 DSSE verification over the PAE encoding, done-check glob, positive control, directive citation.
`directives/build_index.py`	374	Builds the obligation index; Section A parsed verbatim from the rendering, status derived from the live campaign.
`tools/directive_coverage.py`	176	Joins the directive index against the campaign and the fire log; exits nonzero on an unowned obligation or an unresolvable citation.
`tools/campaign_invariants.py`	164	The invariants the campaign must always hold, chiefly: no human- or lab-owned item is ever pending.
`lib/write_summary.py`	87	Appends one fire-summary record. Agent JSON arrives on stdin because passing it as argv once tripped execve E2BIG and lost every verification field for that fire.
`directives/render_directive.py`	82	Deterministic .docx to Markdown, so the document hash is stable.
`tools/check_unbound.py`	32	Flags variables used before assignment. Written after that exact bug aborted a fire, and it has since caught a reintroduction.
`mirror-log.sh`	9	Pushes the fire log to xbabe2 so the evidence tree carries its own operations record.

KEY PROMPT CLAUSES

What the agent is actually told

The prompt is the loop's real specification. Four clauses do most of the work.

1. FALSIFICATION FIRST

> *Your job this hour is to try to BREAK the assigned claim. Design the attack most likely to refute it … a harder held-out split, a distribution-shift or edge-of-envelope probe, re-derivation from an independent published baseline, physical admissibility auditing of the underlying labels.*

A refutation is a first-class success. This is the single most important line in the file: it inverts the incentive that would otherwise make every fire report a win.

2. THE POSITIVE CONTROL

> *A SURVIVAL NOW REQUIRES A POSITIVE CONTROL. Naming a refutation criterion is not enough … Run your attack twice: once against the real input (it must not fire) and once against a copy with a defect of the KIND the attack exists to catch (it MUST fire). An attack that cannot detect a defect you planted yourself cannot license a survival.*

Added after a fire proved the old bar was gameable — it built an attack whose refutation criterion was algebraically unreachable and passed every mechanical check.

3. THE DIRECTIVE CITATION

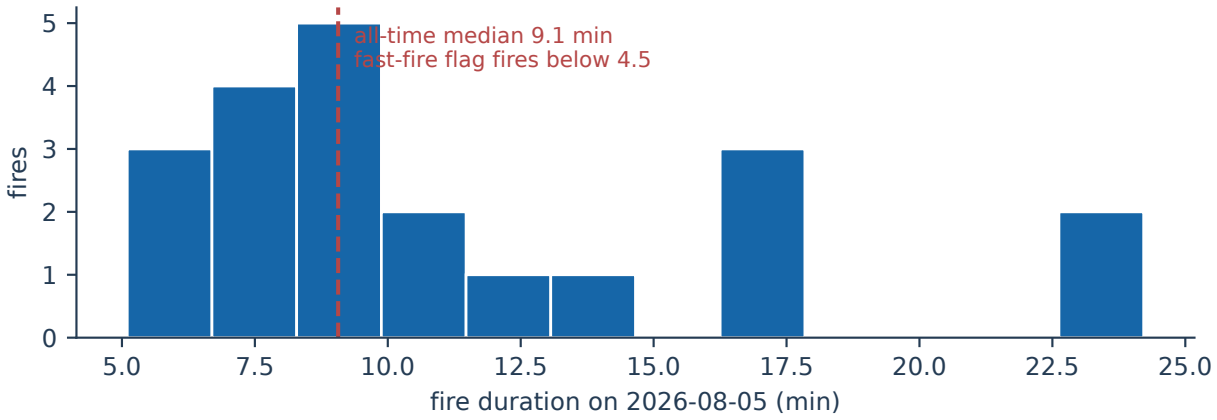
> *If the obligation carries a refutation_criterion, restate it VERBATIM in the receipt and state plainly whether the observation it names occurred. That criterion was pre-committed by the founder before the attack ran, which is the whole point — you may not soften, reinterpret or substitute it.*

4. SUPERSEDE, NEVER REWRITE

> *Supersede-never-rewrite; sign only with the pinned evaluator seed and self-verify DSSE before declaring; done receipt ONLY into the done/ glob when the criterion is met, signed checkpoints into attempts/ otherwise.*

An original is never edited. A correction is a new signed artifact naming what it supersedes by hash. That is why the record can contain its own errors without losing its integrity.

COST



238 fires to date; median 9m 4s, longest 48m. One fire an hour, one at a time, with a pinned model.

LIMITS

What this machinery still cannot do

- **It cannot make a survival mean as much as a refutation.** A positive control shows an attack *can* fire. It does not show the attack was the right one.
- **It cannot verify intent.** The citation gate proves a receipt names the obligation it served. Whether the work served the founder's meaning is a human judgement, and the audit item that asks it has not run.
- **It cannot close a human or lab obligation.** 10 human and 11 lab obligations are structurally out of reach. A low closure rate there is honest, not a shortfall.
- **It cannot produce measured data.** Every gate in this document governs reasoning about existing evidence. None of it creates a coating, a panel, or a measurement.
- **Two ledger rows carry corrupt timestamps** and always will. The generating bug is fixed; the ledger is append-only, so the rows are permanent and every consumer strips the prefix. Repairing them would make the chain look cleaner while destroying the tamper-evidence that makes it worth reading.

EFFICACY, MEASURED

Fires per closed item. 1.14 across 56 closed items. The criteria were rewritten today specifically to raise this — an item that closes on its first fire usually had a criterion satisfiable by reporting rather than by changing anything. Expect this number to rise; if it does not, the rewrite failed.

HOW TO CHECK ANY OF THIS

The loop's configuration is a git repository. Every change in this document is a commit with its reasoning in the message, including the correction of the false one. Run `tools/campaign_invariants.py` and `tools/directive_coverage.py` against the live files; both exit non-zero on violation. The evidence tree is on xbabe2 and every receipt verifies independently against the pinned key.